Group 19

Java O-O Program Visualizer/Animator

Test Plan Document

Names:
Prottasha Deb
Angelina Martis
Arya Patel
Aiden Stensrud

Section:
CSE 4101 Section 1

Date: 09/28/2026

TABLE OF CONTENTS

1. INTRODUCTION **3**

2. REFERENCES **3**

3. TEST CLASSES & OVERALL TEST CONDITIONS **3**

1. Test Item Identifiers 3
2. Test Classes 4
3. Entry Conditions 4
4. Test Data and Independence 4

4. PARSER & MEMORY STATE TEST CASES **4**

1. Primitive Declarations and Assignments 4
2. Object Allocation and Constructor Arguments 5
3. Reference Relationships 5
4. Invalid and Unsupported Source 5

5. INTEGRATION ANIMATION & INTERFACE TESTS **5**

1. Complete Source-to-Diagram Workflow 5
2. Forward Stepping 5
3. Animation Synchronization 5
4. Diagram Layout and Visual Conventions 6
5. Source Input and Interface Controls 6
6. Transition Speed Control 6

6. ENVIRONMENT MILESTONES & RESPONSIBILITIES **6**

1. Test Environment 6
2. Milestone 1: Mockups and Preparation 6
3. Milestone 2: Parser and Generator Integration 7
4. Milestone 3: Full System Evaluation 7
5. Responsibilities and Communication 7
6. Training Schedule and Cost 7

7. RISKS QA AND DELIVERABLES **7**

1. Layout Overlap 7
2. Quality Assurance and Regression 8
3. Test Deliverables 8

1. INTRODUCTION

The purpose of testing is to determine whether supported Java programs produce correct and understandable memory diagrams and whether the interface keeps those diagrams synchronized with execution controls. A successful parse alone is insufficient: the resulting values, objects, references, and method frames must also match the expected program behavior.

Testing covers the Java lexer, syntax, and semantic analyzer, memory instruction interpreter, diagram generator, animator, and input controls. Component tests isolate behavior within these parts. Integration tests examine the information passed between them. System tests exercise the complete workflow, while visual tests assess labels, shapes, arrows, and layout.

2. REFERENCES

[1] Holliday, Luginbuhl, "A Guide to Using Memory Diagrams," Western Carolina University.

https://agora.cs.wcu.edu/~holliday/teaching/LectureNotes/150/MemoryDiagramGuide.pdf

[2] Holliday, Luginbuhl, "Using Memory Diagrams When Teaching a Java-Based CS1," Proceedings of the 41st Annual ACM Southeast Conference.
https://www.researchgate.net/profile/Mark-Holliday-2/publication/242406239_Using_Memory_Diagrams_When_Teaching_a_Java-Based_CS1/links/0c96052b8ef4e9e9b7000000/Using-Memory-Diagrams-When-Teaching-a-Java-Based-CS1.pdf

[3] Holliday, Luginbuhl, "CS1 Assessment Using Memory Diagrams," Proceedings of 35th SIGCSE Tech Symposium on CS Education.
https://dl.acm.org/doi/pdf/10.1145/971300.971373

3. TEST CLASSES & OVERALL TEST CONDITIONS

1. Test Item Identifiers

- Java lexer and syntax analyzer. Check recognition of supported input and identification of malformed or unsupported input. Inspect the resulting program representation or generated instructions when the implementation makes them available.
- Memory instruction interpreter and diagram generator. Check the values and reference relationships derived from the program, the creation of represented objects, and the contents of active method frames. Compare the generated state with an independently prepared expected state.
- Memory diagram animator and renderer. Check state transitions, diagram shapes, labels, arrow targets, and the consistency of the completed animation with its selected execution state.
- Input interface and controls. Check source entry and loading, available navigation controls, code highlighting, and the organization of the editor and diagram areas. Test transition-speed controls only if they are included in the agreed interface scope.

2. Test Classes

Parser syntax classes include supported valid programs, malformed programs, and valid Java that uses unsupported constructs. Keep these categories separate so that rejection of an unsupported feature is not mistaken for rejection of a supported program.

Memory state classes include primitive declarations and assignments, allocated objects, references, string literals, and method calls. Use small examples to isolate one concept before combining concepts. Include more complex supported states to reveal problems that are not visible in a one-variable diagram.

Animation and control classes include advancing through execution and matching the selected state to the highlighted source location. Layout classes examine whether objects, fields, frames, labels, and reference arrows remain distinguishable.

3. Entry Conditions

Before a test cycle begins, identify the build or commit, the operating system, the supported Java subset, and the diagram conventions. Prepare the input files and their expected states. Confirm that the application launches and that a minimal supported example reaches a visible diagram. If this initial check fails, record the failure and block tests that depend on that path.

We must agree whether a highlighted line identifies the operation about to execute or the operation just completed. A line may contain several operations; the expected trace must follow the application's documented step granularity rather than assume exactly one state per source line.

4. Test Data and Independence

Store source examples and expected results with stable identifiers in the project repository. A reviewer should check expected states independently of the generated display. Otherwise, copying the application's output into an expected diagram could preserve the same defect in both the implementation and the test.

4. PARSER & MEMORY STATE TEST CASES

Run these cases with small, deterministic examples within the agreed subset. Record the exact source used and compare each relevant state, not only the final image. Cases involving an unimplemented but required construct remain blocked until the prerequisite is available.

1. Primitive Declarations and Assignments

- Procedure: Enter a supported program that declares an integer with value 2, then assigns it value 5. Analyze the program and step through both operations.
- Expected result: Analysis succeeds. The declared variable has the correct name and value in each state; the assignment changes the value to 5 in the appropriate frame.

2. Object Allocation and Constructor Arguments

- Procedure: Create one instance of a supported class using a parameterized constructor. Choose an argument whose effect on an instance field is easy to verify, then inspect the constructor and the resulting object state.
- Expected result: The trace correctly represents the object and its initialized fields. Constructor parameters and frame behavior match the supported execution model, and the variable refers to the created object.

3. Reference Relationships

- Procedure: Create an object and assign its reference to a second variable. Inspect both reference boxes. If field mutation is supported, change a field through one reference and inspect the target.
- Expected result: Both arrows identify the same object. The diagram does not duplicate the object merely because two variables refer to it. A supported mutation updates that shared object.

4. Invalid and Unsupported Source

- Procedure: Run one program with a deliberate syntax error and a separate valid Java program using a documented unsupported construct. Record the response to each.
- Expected result: Neither case crashes the application nor presents a successful trace for rejected input. Feedback identifies the problem as far as the analyzer supports; diagnostic wording and location expectations must be agreed before execution.

5. INTEGRATION ANIMATION & INTERFACE TESTS

1. Complete Source-to-Diagram Workflow

- Procedure: Enter or load a supported example that combines primitive values, an object reference, and a method call. Analyze it and navigate through the available states.
- Expected result: The parser, interpreter, generator, and renderer complete the workflow. Every checked diagram agrees with its expected state, and the highlighted source location follows the agreed stepping convention.

2. Forward Stepping

- Procedure: Advance through at least three available states, capturing their values and relationships.
- Expected result: Navigation does not accumulate duplicate nodes or otherwise alter the represented program state.

3. Animation Synchronization

- Procedure: Observe a transition that changes a value or reference. Inspect the starting state, the completed transition, and the source highlight.

- Expected result: The completed animation agrees with the selected state. Highlighting and diagram changes follow the same step convention. Animation does not suggest a state change that the interpreted program did not make.

4. Diagram Layout and Visual Conventions

- Procedure: Render a small example and a larger supported example with multiple variables and references. Compare shapes and labels with the agreed conventions and inspect each arrow target.
- Expected result: Labels remain readable, nodes do not obscure one another, and arrows have identifiable origins and targets. Reference variables use rectangles. Confirm the final object and string shapes before applying shape-specific pass/fail rules.

5. Source Input and Interface Controls

- Procedure: Enter and edit code, load a source file, clear the editor, and use the available toolbar controls. Check the source display and diagram area after each operation.
- Expected result: Text and line numbering reflect the input. Controls perform their labeled actions, and interface elements remain usable without hiding essential information. Source edits must not be mistaken for the source of an older trace.

6. Transition Speed Control

- Procedure: If the speed adjustment is approved and implemented, replay the same supported transition in two available settings and compare the resulting states. Otherwise, record the case as not applicable with a scope explanation.
- Expected result: The selected setting changes transition timing while preserving the state sequence and final values. No timing threshold is assumed until the control's intended behavior is documented.

6. ENVIRONMENT MILESTONES & RESPONSIBILITIES

1. Test Environment

The planned operating systems are Windows 11 and macOS. Record the exact macOS version and Windows build used. Also record the machine's processor, memory, display resolution, scaling, application build, runtime, and relevant graphics or interface library versions. These details support reproduction without inventing minimum hardware requirements.

Use the same source examples and expected states on both platforms. Compare program functionality and usability rather than requiring identical pixel dimensions or system font rendering. Run the main workflow on each platform and investigate any differences in source loading, layout, input controls, or animation.

2. Milestone 1: Mockups and Preparation

Prepare this plan, initial source examples, and manually checked expected diagrams. Review parser behavior through available prototypes and inspect visual

mockups. Document gaps in the language subset and diagram conventions. This milestone establishes test readiness; it does not assume that an integrated parser or animator is already complete.

3. Milestone 2: Parser and Generator Integration

Execute the parser and memory-state cases supported by the current build, then test the integration of source analysis with a simple diagram generator. Record defects and rerun affected cases after corrections. Use more involved supported examples to expose reference-target and layout problems before animation is integrated.

4. Milestone 3: Full System Evaluation

Test the combined parser, interpreter, generator, animator, and interface. Execute navigation, synchronization, visual, and platform checks. Test speed adjustment if it remains in the agreed scope. Prepare a final results summary identifying passes, failures, blocked cases, accepted limitations, and evidence for the demonstrated build.

5. Responsibilities and Communication

Assign a test owner, expected-result reviewer, defect owner, and cycle reviewer before each cycle. Roles may be shared among team members, but every test and defect needs a named owner in its record. No individual assignments are fixed by this plan.

Use messages, Google Meet, and email for coordination. Maintain shared documents in Google Docs and source code and test examples in GitHub, as planned. Capture decisions about scope, expected diagrams, and defect acceptance in a durable project record so that informal discussions do not become the only source of test expectations.

6. Training Schedule and Cost

Training includes compiler construction, memory diagram conventions, and visual testing. Use small examples to practice reading traces and identifying incorrect references. Testing follows the three milestones; dates and effort estimates remain to be assigned. The stated financial budget is $0, assuming access to existing machines and tools. Team labor still requires scheduling even when no purchase is planned.

7. RISKS QA AND DELIVERABLES

1. Layout Overlap

Overlap is a high-severity risk. Crowded states can hide values or make reference targets ambiguous even when the underlying interpretation is correct. Test progressively larger supported examples and retain screenshots of problematic arrangements. Automatic spacing is a proposed mitigation, not evidence that the risk has been resolved.

After a layout correction, rerun the failing example and nearby simpler cases. Confirm that moving nodes has not broken arrow endpoints, labels, or frame grouping. If the selected layout cannot clearly display a required state, record a defect or an agreed scope limit rather than treating the underlying data alone as sufficient.

2. Quality Assurance and Regression

Review test inputs, expected diagrams, and results for consistency with the agreed requirements. Link each case to a requirement or a named feature, and identify any uncovered features. After a change, rerun cases affected by that change plus a complete source-to-diagram workflow. Broaden regression coverage when shared state or rendering behavior may have changed as well.

3. Test Deliverables

Deliver the maintained test plan, reusable Java input files, manually checked expected states or diagrams, execution records, and defect reports. Include automated test scripts or executables where developed, with instructions for running them. Screenshots and recordings should identify the test case and build they document. The final test summary should state the tested build, environments, supported scope, status counts, unresolved defects, and accepted restrictions. Distinguish tests that were planned from tests actually executed. Keep the evidence in the shared project records so that later changes can be compared against the same examples and expectations.